Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Insertion sort: akış diyagramı, kaydırma mantığı, en iyi ve en kötü durum.
- Bir sayının asal olup olmadığını bulma problemi ve adım adım iyileştirilmesi.
- Eratosthenes Kalburu (Sieve of Eratosthenes) ile n'e kadar tüm asal sayıları bulma.
- Doğrusal arama (linear/sequential search) ve ikili arama (binary search). Azalt ve yönet (decrease and conquer) yaklaşımı.
- Hash ile aramaya kısa değinme.
- Temel işlemler (basic/primitive operations) sayılarak zaman karmaşıklığının hesaplanması.
- Big O notasyonu: sabitlerin ve küçük dereceli terimlerin atılması.
- En iyi, ortalama ve en kötü durum (best, average, worst case) örnekleri: doğrusal arama, en küçük ve en büyük elemanı bulma.
- Büyüme dereceleri: 1, $\log n$, n, $n \log n$, n^2, n^3, 2^n, $n!$.
- Asimptotik notasyon: Big O (üst sınır), Big Omega (alt sınır), Big Theta (sıkı sınır).
- Sözde kod (pseudo code) ile en büyük elemanı bulma ve işlem sayımı.
- Teorik analiz ile deneysel (ampirik) ölçümün karşılaştırılması.

Hocanın Özellikle Vurguladığı Kısımlar

- Birden fazla koşul varsa sınır kontrolü ilk sırada yazılmalı. Insertion sort'ta önce `j >= 0`, sonra `a[j] > tmp` kontrol edilir. Ters sırada `a[-1]`'e erişilir ve segmentation fault oluşabilir.
- İki koşulun birlikte sağlanması gerekiyorsa AND, birinin yetmesi yeterliyse OR kullanılır.
- Karşılaştırma yapan sıralama algoritmaları iç içe iki döngüye ihtiyaç duyar.
- Bir probleme önce kaba kuvvet (brute force, naive) çözümle başlayın ve doğru çalışan bir çözüm elde edin. Sonra işlem sayısını, ardından bellek kullanımını azaltmaya çalışın.
- İşlem sayısını azaltmak çoğu zaman ek bellek gerektirir. Bellek kısıtı varsa (gömülü sistemler, çok büyük veri) daha yavaş ama az bellekli çözüm seçilebilir.
- Big O'da sabit sayılar ve küçük dereceli terimler yazılmaz.
- Karmaşıklığı asıl belirleyen döngülerdir. Ardışık döngüler toplanır, iç içe döngüler çarpılır.
- Döngü değişkeni ikişer ikişer ya da ikiye katlanarak ilerliyorsa dikkatli olun. Bu durumda karmaşıklık logaritmik olabilir.
- Üst sınırı en yakın değerle verin. n^3'lük bir algoritma için $O(n^4)$ yazmak teknik olarak yanlış değil, ama tercih edilmez.
- Bu dönem sorumlu olunan kısım Big O'dur: bir akış diyagramının ya da sözde kodun karmaşıklığını Big O ile ifade edebilmek. Ortalama durum, Omega ve Theta Algoritma Analizi dersinde işlenecek.
- Sözde koddan uzak durmayın. Konuşma diline yakın, sade bir gösterimdir.
- Hashing bu derste işlenmeyecek, ama kısaca okumak faydalı olur.

Kısa Tekrar Notları

- Insertion sort: ikinci elemandan başlanır. Eleman `tmp`'ye alınır, ondan büyük elemanlar birer sağa kaydırılır, `tmp` açılan yere yazılır.
- Insertion sort'ta swap yoktur, yalnız kaydırma vardır.

- Insertion sort en iyi durum (sıralı dizi): $n-1$ karşılaştırma, $O(n)$. En kötü durum (ters sıralı dizi): $O(n^2)$.
- Insertion sort küçük ve sıralıya yakın dizilerde selection ve bubble sort'tan daha çok tercih edilir.
- Asallık testi iyileştirmeleri: $x-1$'e kadar bölmek, $x/2$'ye kadar bölmek, $\sqrt{x}$'e kadar bölmek, 2'yi ayrı kontrol edip yalnız tek sayılara bölmek.
- Eratosthenes Kalburu: n boyutlu dizi, 0 = asal adayı, 1 = asal değil. Her asal i için $i \cdot i$'den başlanır, $2i$ adımla işaretlenir. Dış döngü $\sqrt{n}$'e kadar gider. Dezavantajı ek bellek kullanmasıdır.
- Binary search: yalnız sıralı dizide çalışır. Her adımda aranan aralığın yarısı elenir, $O(\log n)$. $n = 1024$ için en fazla 10 adım.
- Hash ile arama ortalama durumda $O(1)$.
- Sırasız dizide doğrusal arama: en iyi $O(1)$, en kötü $O(n)$.
- Sırasız dizide en küçük ve en büyüğü bulma: en iyi durumda da $O(n)$, çünkü bütün elemanlara bakmak gerekir.
- Sıralı dizide en küçük ve en büyük: ilk ve son eleman, her durumda $O(1)$.
- $4n + 5 \Rightarrow O(n)$, $n^2 + 100n \Rightarrow O(n^2)$, $n^2 + 5n + 3 \Rightarrow O(n^2)$.
- Bir kümenin tüm alt kümelerini denemek 2^n iş demektir. $n = 6$ için 64 alt küme, $n = 100$ için çözülemez.

Detaylı Açıklamalar

Insertion sort

- İskambil kâğıdı sıralamaya benzer. Her eleman, solundaki sıralı kısım içinde doğru yerine yerleştirilir.
- Örnek: 5, 1, 4, 3, 2, 7, 6.
  - 1 alınır, 5 sağa kayar: 1, 5.
  - 4 alınır, 5 sağa kayar: 1, 4, 5.
  - 3 alınır, 5 ve 4 sağa kayar, 3 açılan yere yazılır: 1, 3, 4, 5.
  - Aynı işlem 2, 7 ve 6 için tekrarlanır.

```
for (i = 1; i < n; i++) {
    tmp = a[i];
    j = i - 1;
    while (j >= 0 && a[j] > tmp) {
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = tmp;
}
```

- `while`'daki iki koşul AND ile bağlı. `j` 0'ın altına inerse ya da `a[j]` artık `tmp`'den büyük değilse döngü biter.
- Sıralı dizi (1, 2, ..., 7): her turda tek karşılaştırma yapılır, iç döngüye hiç girilmez. Toplam $n-1$ karşılaştırma.
- Ters sıralı dizi: her eleman en başa kadar kaydırılır, $O(n^2)$.
- Avantajları: swap yerine kaydırma yapılır ve gerektiği kadar eleman kaydırılır. Sıralıya yakın dizilerde çok hızlıdır.

Asal sayı problemi

- Asal sayı (prime number): yalnız 1'e ve kendisine tam bölünebilen sayı.
- Asal sayılar rastgele sayı üretiminde ve kriptolojide kullanılır. Şifrelemede büyük asal sayılar önemlidir.
- Kaba kuvvet çözüm: x'i 2'den $x-1$'e kadar bütün sayılara bölmeyi denemek. Örneğin 37 için 2'den 36'ya kadar.
- İyileştirme 1: bölenler çiftler halinde gelir. 36 için (2, 18), (3, 12), (4, 9). Yarıya kadar bakmak yeter.
- İyileştirme 2: aslında $\sqrt{x}$'e kadar bakmak yeter. 37 için 2-6 (7'yi de kontrol edebilirsiniz) yeterli.

- İyileştirme 3: önce 2'ye bölünüp bölünmediğine bakılır. Sayı tekse yalnız tek bölenler denenir (3, 5, 7, 9, ...). Çünkü 2'ye bölünmeyen bir sayı 4'e, 6'ya da bölünmez.

```
if (x % 2 == 0)
    printf("not a prime number");
else {
    sinir = sqrt(x);
    i = 3;
    while (i <= sinir && x % i != 0)
        i = i + 2;
    if (i > sinir)
        printf("prime number");
    else
        printf("not a prime number");
}
```

- 101 için $\sqrt{101} \approx 10$. Önce 2, sonra 3, 5, 7, 9 denenir ve sayı asal bulunur.
- Döngüden sınırı aşarak çıkılmışsa hiçbir bölen bulunmamıştır, sayı asaldır.

Eratosthenes Kalburu (Sieve of Eratosthenes)

- Amaç: n'nin asal olup olmadığını bulurken n'e kadar bütün asal sayıları da bulmak.
- n boyutlu bir dizi kullanılır. Bütün gözler başta 0'dır, yani her sayı asal adayıdır. Asal olmadığı anlaşılan sayılar 1 ile işaretlenir.
- Adımlar:
  - 2 özel bir sayıdır. 4'ten başlayarak ikişer ikişer ilerlenir: 4, 6, 8, 10, ... 1 yapılır.
  - $nn = \sqrt{n}$ hesaplanır. Dış döngü 3'ten nn'e kadar, ikişer artarak gider.
  - `a[i]` hâlâ 0 ise i asaldır. $i \cdot i$'den başlanır, $2i$ adımla n'e kadar bütün sayılar 1 yapılır.
  - Sonunda 2'den başlayarak değeri 0 kalan sayılar asaldır (0 ve 1 asal değildir).

i	Başlangıç ($i \cdot i$)	Adım ($2i$)	İşaretlenenler
3	9	6	9, 15, 21, 27, ...
5	25	10	25, 35, 45, ...
7	49	14	49, 63, 77, ...

- Neden $i \cdot i$'den başlanır? Daha küçük katlar zaten daha küçük asallar tarafından işaretlenmiştir. Örneğin 3 için 6 ve 12, 2 tarafından işaretlendi.
- Neden $2i$ adım? i adımla ilerlenseydi çift katlara da uğranırdı. Örneğin 25'ten 30'a gidilirdi, ama 30 zaten 2 ile işaretlendi.
- 36 için $nn = 6$, yalnız 3 ve 5 incelenir. 101 için $nn = 10$, dış döngü 3, 5, 7 ve 9 değerlerini alır. 9 zaten 3 tarafından işaretlendiği için atlanır, katları yalnız 3, 5 ve 7 için işaretlenir.
- Dezavantajı: diğer yöntemler sabit bellekle çalışırken bu yöntem n boyutlu ek bir dizi kullanır.

Arama algoritmaları

- Doğrusal arama: elemanlar tek tek kontrol edilir. En kötü durumda n karşılaştırma.
- İkili arama: yalnız sıralı dizide uygulanır. Azalt ve yönet (decrease and conquer) yaklaşımının örneğidir. Bu yaklaşım böl ve yönet (divide and conquer) başlığı altında anılır.
- $n = 1024$ için doğrusal arama 1024, ikili arama yaklaşık $\log_2 1024 = 10$ karşılaştırma yapar. Bir milyon eleman için ikili arama yaklaşık 20 adım sürer.
- Tercih sırası: mümkünse $O(\log n)$, değilse $O(n)$, en azından $O(n^2)$'den küçük.
- Hash: aranan değerden bir hesapla dizideki gözün yeri bulunur, ortalama bir adımda erişilir. Bu ders kapsamında işlenmeyecek.

İkili arama akışı

- `first = 0`, `last = n - 1`, `found = 0`.
- `first <= last` ve `found == 0` olduğu sürece:

  - `mid = (first + last) / 2` (tam sayı bölmesi).
  - `a[mid] == x` ise `found = 1`.
  - Değilse ve `x < a[mid]` ise `last = mid - 1` (sol yarı).
  - Değilse `first = mid + 1` (sağ yarı).
- Döngü bitince `found` 1 ise bulundu, 0 ise "not found" yazılır.
- Derste 8 elemanlı bir dizide 4 aranıyor (dizide yok). Adımlar:

Adım	first	last	mid	Karşılaştırma
1	0	7	3	`a[3] = 6`, $4 < 6$, sola git
2	0	2	1	`a[1] = 3`, $4 > 3$, sağa git
3	2	2	2	`a[2] = 5`, $4 < 5$, sola git
4	2	1	-	`first > last`, döngü biter, bulunamadı

- Her adımda ya `first` artar ya `last` azalır. İkisi birbirini geçince arama biter.
- `found` bayrağı yerine çıkış koşulu doğrudan döngü koşuluna yazılabilir.

Temel işlemler ve Big O

- Temel işlemler: atama, aritmetik işlemler (toplama, çıkarma, çarpma, bölme), karşılaştırma, değişken tanımlama, bellek erişimi.
- İşlemlerin işlemci döngüsü (cycle) sayısı farklı olsa da her biri bir birim iş sayılır. Önemli olan kaç kez yapıldıklarıdır.
- Derste bir döngü örneğinde: 3 değişken tanımı, $n + 2$ atama, n karşılaştırma, n artırma, n çarpma. Toplam $4n + 5$, yani $O(n)$.
- n sonsuza giderken sabit katsayılar ve küçük dereceli terimler önemsizleşir.
- Ardışık 10 tek döngü $O(10n) = O(n)$ verir. İç içe iki döngü $O(n^2)$, üç döngü $O(n^3)$ verir.
- Girdi n ile ifade edilmiyorsa ilgili değişkenler kullanılır: $O(k)$, $O(m)$, $O(n \cdot m)$ gibi.
- Girdinin sıralı ya da sırasız olması da karmaşıklığı doğrudan etkiler.

En iyi ve en kötü durum örnekleri

Problem	Girdi	En iyi durum	En kötü durum
Doğrusal arama	Sırasız	$O(1)$ (ilk eleman)	$O(n)$ (sonda ya da yok)
Doğrusal arama	Sıralı	$O(1)$	$O(n)$
En küçük ve en büyük	Sırasız	$O(n)$	$O(n)$
En küçük ve en büyük	Sıralı	$O(1)$	$O(1)$

- Sıralı dizide ilk eleman en küçük, son eleman en büyüktür. Bunun için sıralama yönünün bilinmesi gerekir.

Büyüme dereceleri

- $O(1)$: en sevilen derece, işlem tek adımda biter.
- $O(\log n)$: çok yavaş büyür. Örnek: ikili arama.
- $O(n)$: doğrusal zamanda çözüm çok iyidir.
- $O(n \log n)$: heap sort, merge sort, quick sort gibi algoritmalar. n^2'ye göre tercih edilir.
- $O(n^2)$ ve $O(n^3)$: n bir milyon olduğunda 10^{12} mertebesinde iş, sorun çıkmaya başlar.
- $O(2^n)$ ve $O(n!)$: $n = 100$ için en güçlü bilgisayarlar bile makul sürede çözemez.
- Örnek: {1, 3, 2, 8, 6, 4} kümesinden toplamı 12 olan sayıları bulmak. Tüm alt kümeleri denemek 2^n iş demektir. $n = 6$ için 64, $n = 100$ için çözülemez. Bu tür kombinatorik problemler için özel yaklaşımlar gerekir.

Asimptotik notasyon

- Big O: üst sınır (upper bound). En çok kullanılan notasyon.
- Big Omega: alt sınır (lower bound). Daha az kullanılır.
- Big Theta: sıkı sınır (tight bound). Fonksiyon hem üstten hem alttan aynı büyüme derecesiyle sınırlanır.
- Üst sınır en yakın makul değerle verilmelidir.
- Omega ve Theta bu dersin konusu değil.

Sözde kod ve en büyük eleman

- Sözde kodda atama için ok (←), döngü için `for i ← 1 to n do` gibi gösterimler kullanılabilir.

```
max ← A[0]
for i ← 1 to n - 1 do
    if A[i] > max then
        max ← A[i]
return max
```

- İşlemler sayılır: `A[0]` erişimi ve atama, döngü değişkeni atamaları, n kez döngü kontrolü, n kez `A[i] > max` karşılaştırması. Sabitler atılınca $O(n)$.
- Tek döngü olduğuna bakarak da $O(n)$ denebilir.

Teorik ve deneysel analiz

- Teorik analiz yazılım ve donanımdan bağımsızdır. Kod çalıştırmadan, temel işlemler sayılarak iki çözüm objektif olarak karşılaştırılır.
- Çok uzun kodlarda (40-50 bin satır) teorik analiz zordur. Bu durumda iki kod aynı donanım ve yazılımda çalıştırılır.
- Farklı girdi boyutları ($n = 10, 100, 1000, 10000$) için her ölçüm 10-20 kez tekrarlanıp ortalama alınır. Girdi boyutu-süre grafiği çizilir.
- Küçük girdilerde sabitler nedeniyle sonuç tersine dönebilir. Örneğin $100n + n^2$'deki 100 küçük n'de etkilidir. Hep küçük girdilerle çalışıyorsanız o durumda daha hızlı olan algoritmayı seçin.

Ders sonrası notlar

- Hoca bir yerde "doğrusal aramanın en kötü durum karmaşıklığı O(1)" dedi. Bu bir dil sürçmesi. Doğrusal aramada en iyi durum $O(1)$, en kötü durum $O(n)$'dir.
- Asallık testinde hoca döngü sonunda "i eşit x ise prime" diye bir kontrol anlattı, ardından "sınır artı bir" ifadesini kullandı. Doğru kontrol, döngüden sınırı aşarak çıkılıp çıkılmadığıdır (`i > sinir`). Hiçbir bölen bulunmadan döngü bitmişse sayı asaldır.
- Asallık testinde 2 ayrıca ele alınmalıdır. `x % 2 == 0` kontrolü 2'yi de asal değil sayar, oysa 2 asaldır. 1 de asal değildir.
- Örnekte geçen 987.972 çift sayıdır, asal değildir. Hoca sayıyı yalnız büyük sayı örneği olarak verdi.
- Algoritmanın adı Eratosthenes Kalburu'dur (Sieve of Eratosthenes).
- Hoca "n log n karmaşıklığında quick sort" dedi. Quick sort ortalama durumda $O(n \log n)$, en kötü durumda $O(n^2)$'dir. Heap sort ve merge sort her durumda $O(n \log n)$'dir.
- Hoca insertion sort'un en iyi durumda "n tane karşılaştırma" yaptığını söyledi. Sıralı dizide her eleman (ikinciden başlayarak) yalnız bir kez karşılaştırılır, toplam $n-1$ karşılaştırma yapılır. Karmaşıklık yine $O(n)$'dir.
- Hoca kalbur örneğinde 101 için "3, 5, 7 ve 9'un peşinden gidecektik" dedi. 9 daha önce 3'ün katı olarak 1 ile işaretlendiği için `a[9]` kontrolünden geçemez, katları işaretlenmez.
- Hoca kalbur sonunda "birden n'e kadar 0 gördüğünüz sayıları yazdırın" dedi. 0 ve 1 asal değildir ve dizide 0 olarak kalır. Yazdırma 2'den başlamalıdır.
- Hoca ikili aramada 12 aranırken sağ yarıya geçişi anlatırken "mid değerine 1 koyacaktık" dedi. Doğrusu `first = mid + 1 = 4` atamasıdır, `mid` değişmez ve sonraki adımda yeniden hesaplanır.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.